

Stress Test Example for Omninet AI Project

Test Scenario: Payment Processing Under Extreme Load

Objective: Verify how the Payment Processor microservice and its supporting infrastructure behave when subjected to extreme conditions beyond normal operational limits. Specifically, we want to identify the breaking point and observe failure modes.

Test Description

We simulate a **flash mob scenario** where hundreds of kiosks simultaneously attempt to process payments during a peak hour (e.g., New Year's Eve at an airport). The load is gradually increased beyond the rated capacity until the system begins to fail.

Test Environment Setup

- **Test Tool:** JMeter or Locust (to simulate concurrent kiosk payment requests)
 - **Target:** Payment Processor microservice (via API Gateway)
 - **Monitoring:** CPU, memory, response times, error rates, database connection pool usage
 - **Baseline Rated Capacity:** 1,000 concurrent payment requests per second (based on SysRD 040-020-010)
-

Test Execution Steps

Step	Action	Expected Behavior	Actual Observation (Example)
1	Ramp up to 800 requests/sec (80% of rated capacity)	All requests processed within 2 seconds; error rate <1%	✅ Pass
2	Increase to 1,000 requests/sec (rated capacity)	Response time may increase slightly but stays under 3 seconds; error rate <2%	✅ Pass
3	Increase to 1,500 requests/sec (150% of rated capacity) - Stress condition	System begins to show strain: response times spike, some requests timeout, database connection pool saturates	⚠️ Monitor
4	Hold at 1,500 requests/sec for 5 minutes	System should fail gracefully : reject new requests with proper HTTP 503 (Service Unavailable) rather than crashing; already accepted transactions complete successfully	🔍 Observe failure mode
5	Gradually reduce load back to 800 requests/sec	System should recover automatically: response times return to normal, error rate drops	✅ Pass if recovery occurs

Stress Conditions Applied

Stress Element	How It Was Stressed	Omninet Context
Volume (rate of data flows)	1,500 payment requests/sec (50% above rated)	Simulates a sudden surge of users at a major transportation hub
Capacity (resource depletion)	Fill database connection pool to 100%	Payment Processor cannot obtain database connections for new requests
Error combinations	Inject network latency between Payment Processor and Bank Authorization Stub	Simulates slow response from external bank API

Concrete Example of a Stress-Induced Failure

During the test at 1,500 requests/sec, the following occurs:

1. **Database connection pool** (max 100 connections) becomes fully occupied.
 2. New payment requests wait for a connection; wait time exceeds 5 seconds.
 3. **Payment Processor** begins rejecting requests with HTTP 503.
 4. **Kiosks** receive the 503 error and display: "*Service temporarily unavailable. Please try again in a few moments.*" (graceful degradation).
 5. **Monitoring alert** triggers: "Payment Processor – connection pool exhaustion."
 6. After load reduces, connections are released, and the service recovers.
-

Pass/Fail Criteria

Criteria

System crashes or becomes completely unresponsive

Data corruption occurs (e.g., duplicate transactions)

System rejects requests with proper error messages (503) and recovers automatically

Already accepted transactions are not lost

Why This Stress Test Matters for Omninet

- **Public-facing system:** Airports and malls experience unpredictable surges (holidays, events, flight delays).
 - **Revenue impact:** If the payment system fails, Omninet loses money and user trust.
 - **Safety:** Graceful failure (vs. crash) ensures users understand what happened and can try again.
 - **Requirement validation:** Directly tests SysRD non-functional requirements for reliability and performance under stress.
-

Related Test Types in This Scenario

Test Type	Role in This Example
Volume Testing	Ramping up to 1,000 requests/sec (rated)

Test Type	Role in This Example
Stress Testing	Exceeding rated capacity to 1,500 requests/sec
Capacity Testing	Monitoring resource exhaustion (DB connections, memory)
Reliability Testing	Observing behavior under sustained high load
Recovery Testing	Verifying automatic recovery after load returns to normal